COMP1117 Tutorial 7

Coding Practice (Strings)

November 2026

From concepts to code

You already know

- Strings, indexing and loops
- Useful methods and slicing
- Functions and return values

Today

- Combine string operations
- Write reusable functions
- Explain and test results

What should the function return? Try one example by hand before coding.

Today's plan

Recall	Trace string operations and keep their returned values.
Build	Write small functions that each do one job.
Combine	Use helpers to build a formatted text report.
Test	Compare normal, empty and boundary inputs.

String methods return new values

A string cannot be changed in place. Save or use the value returned by a method.

EXAMPLE

```
message = 'Hello!'
message.lower()
print(message)

quiet = message.lower()
print(quiet)
```

Output

```
Hello!
hello!
```

Calling lower() does not change message. The second call assigns its result to quiet. Inside a function, return the new value so that the caller can use it.

From text to words, and back

split() produces a list of words. A separator's join() method combines them into a string.

EXAMPLE

```
text = '  red\tblue  '
words = text.split()
print(words)
print(' '.join(words))
print(text.split(' '))
```

Output

```
['red', 'blue']
red blue
['', '', 'red\tblue', '', '']
```

With no argument, split() groups whitespace and drops empty pieces. split(' ') splits only at ordinary spaces. The separator in ' '.join(words) is one ordinary space.

Indexing, slicing and repetition

Indexing selects one character. Slicing selects a sequence of characters.

EXAMPLE

```
word = 'Python'
print(word[0], word[-1])
print(word[1:4])
print(word[::-1])
print('*' * 3)
```

Output

```
P n
yth
nohtyP
***
```

The stop index in [1:4] is excluded. A step of -1 reverses the string. Repetition with * builds a new string. Check for an empty string before taking its first character.

Build a result with an f-string

Calculate a value first, then choose how to display it.

EXAMPLE

```
total = 10
count = 3
average = total / count
print(f'Average: {average:.1f}')
print(f'Words: {count}\nTotal: {total}')
```

Output

```
Average: 3.3
Words: 3
Total: 10
```

Use .1f for one decimal place; it does not change average. A newline escape starts a new line. Check for zero words before dividing.

Quick check: trace the methods

PREDICT BEFORE RUNNING

```
original = '  Hi\tTHERE  ! '
pieces = original.lower().replace('!', '').split()
joined = ' '.join(pieces)
```

Predict pieces and joined before running the code. Has original changed?

What would split(' ') do differently?

Quick check: answer

OUTPUT

```
['hi', 'there']
'hi there'
'  Hi\tTHERE  ! '
```

lower() returns lowercase text; replace() removes !; split() makes words; join() puts one space between them. Each step uses the previous result. original is unchanged.

Task 1: Clean up a message

Return a cleaned version of a message.

`normalise_text(text)`

- Lowercase letters; remove ! only.
- Keep other punctuation and digits.
- Collapse whitespace to single spaces.
- No words left? Return an empty string.

EXPECTED RESULTS

```
normalise_text(
  "  PYTHON\tis Fun!!!  "
)

"python is fun"

"one!two" -> "onetwo"
```

Keep punctuation other than !. Try a tab as well as a space.

Task 1: Answer

ONE POSSIBLE SOLUTION

```
def normalise_text(text):
    cleaned = text.lower().replace('!', '')
    return ' '.join(cleaned.split())
```

Why this works

Remove ! before splitting: 'a ! b' then has two words, not three. split() handles surrounding and repeated whitespace; joining with one space gives the required result.

Compare deleting ! with replacing it by a space in 'one!two'.

Task 2: Find the longest word

Return the word count and the first longest word as a tuple.

`word_summary(text)`

- Return (word count, longest word).
- Use split(); punctuation counts.
- Keep original case; first tie wins.
- Empty input: (0, ''). Use a loop.

EXPECTED RESULTS

```
"red blue gold"
(3, "blue")

""
(0, "")
```

Use a loop. What should happen when two words have the same length?

Task 2: trace the running comparison

TRACE THE LOOP

```
longest_seen = ''
for candidate in 'red blue gold'.split():
    if len(candidate) > len(longest_seen):
        longest_seen = candidate
    print(candidate, '->', longest_seen)
```

```
red -> red
blue -> blue
gold -> blue
```

Why >, not >=?

Start with longest = ''. Visit each word and update only when the new word is longer. A tie keeps the earlier word.

Task 2: Answer

ONE POSSIBLE SOLUTION

```
def word_summary(text):
    words = text.split()
    longest = ''
    for word in words:
        if len(word) > len(longest):
            longest = word
    return len(words), longest
```

Why this works

An empty initial longest works even when words is empty. A strict > comparison keeps the first word when lengths tie. Only whitespace separates words, so 'red,' has length four.

Replace > with >= in this demonstration. Which answer changes, and why?

Task 3: Build a text report

Combine the first two functions and return a four-line report.

`text_report(text)`

- Call the two earlier helpers.
- Average: total word length / count.
- No words: 0.0 and (none).
- Return four lines; average uses .1f.

EXPECTED RESULTS

```
Text: python is fun
Words: 3
Longest: python
Average length: 3.7
```

For empty text: zero words, (none) as the longest word, and average 0.0.

Task 3: solution plan

Clean	`normalise_text(text)`	Normalised text
Analyse	`word_summary(cleaned)`	Count and first longest word
Calculate	`Add word lengths in a loop`	Average; avoid dividing by zero
Format	`Use an f-string with .1f`	Return the four-line report

Task 3: Answer (1/2)

CALCULATE FIRST

```
def text_report(text):
    cleaned = normalise_text(text)
    word_count, longest = word_summary(cleaned)
    total_length = 0
    for word in cleaned.split():
        total_length = total_length + len(word)
    average = 0.0
    if word_count > 0:
        average = total_length / word_count
    if longest == '':
        longest = '(none)'
```

Reuse the helpers. Add word lengths in a loop, and divide only when the count is non-zero.

Task 3: Answer (2/2)

CONTINUED INSIDE text_report

```
    return (f'Text: {cleaned}\n'
            f'Words: {word_count}\n'
            f'Longest: {longest}\n'
            f'Average length: {average:.1f}')
```

Adjacent f-strings inside parentheses form one result. Each \n starts the next line; .1f displays one decimal place.

Task 4: Name initials

Return uppercase initials with a full stop after each initial.

`initials(name)`

- Split on whitespace; take each first letter.
- Uppercase each initial.
- Add a full stop after every initial.
- Empty name: an empty string.

EXPECTED RESULTS

```
"Ada Lovelace"
"A.L."

"  alan\tturing "
"A.T."

""
""
```

Input: ASCII letters and whitespace. What does an empty name produce?

Task 4: Answer

ONE POSSIBLE SOLUTION

```
def initials(name):
    result = ''
    for part in name.split():
        result = result + part[0].upper() + '.'
    return result
```

Why this works

Each whitespace-separated name part contributes one letter and one full stop. An empty list of parts leaves result as ''. A list comprehension is an equally valid alternative.

Trace result for 'alan turing'. What happens when the name contains only whitespace?

Task 5: Count one character

Count occurrences of a single character, ignoring letter case.

`count_character(text, target)`

- target has exactly one character.
- Ignore letter case, not punctuation.
- Spaces can be counted too.
- Return an integer; count() or a loop.

EXPECTED RESULTS

```
("Banana", "A") -> 3

("a a", " ") -> 1

("", "x") -> 0
```

Input: ASCII text; target has exactly one character, including a possible space.

Task 5: Answer

ONE POSSIBLE SOLUTION

```
def count_character(text, target):
    return text.lower().count(target.lower())
```

Why this works

The one-character guarantee makes str.count() agree with counting matching characters in a loop. Lowercasing does not change digits, spaces or punctuation.

How could a loop produce the same count? Why lowercase target too?

Task 6: A different palindrome rule

Check a palindrome: ignore case and whitespace, keep punctuation.

`is_palindrome(text)`

- Ignore case and all whitespace.
- Keep punctuation and digits.
- Compare with the reverse.
- Empty result counts as a palindrome.

EXPECTED RESULTS

```
"Never odd or even"
True

"ab,ba!"
False

""
True
```

This rule keeps punctuation. Compare ab!ba with ab,ba!.

Task 6: Answer

ONE POSSIBLE SOLUTION

```
def is_palindrome(text):
    compact = ''.join(text.lower().split())
    return compact == compact[::-1]
```

Why this works

The loop or split/join version must remove all whitespace, not only spaces. Keeping punctuation makes 'ab!ba' a palindrome but 'ab,ba!' not a palindrome.

Try 'a\tbba', 'ab!ba' and 'ab,ba!'. Why does replace(' ', '') miss one of these cases?

Task 7: Mask a username

Hide all but the first character of an email username.

`mask_email(email)`

- Exactly one @; both parts non-empty.
- Keep the username's first character.
- One * for each remaining character.
- Preserve the domain and letter case.

EXPECTED RESULTS

```
"alice@example.com"
"a****@example.com"

"A@HKU.HK"
"A@HKU.HK"
```

The address has exactly one @ and no whitespace; both parts are non-empty.

Task 7: Answer

ONE POSSIBLE SOLUTION

```
def mask_email(email):
    username, domain = email.split('@')
    masked = (username[0]
              + '*' * (len(username) - 1))
    return masked + '@' + domain
```

Why this works

A username of length one needs zero stars, and '*' * 0 is ''. Splitting with one argument is sufficient because the input guarantees exactly one @.

For 'A@HKU.HK', how many stars are needed? Why should we not lowercase the whole address?

Before you leave

Testing checklist

- A normal example
- An empty or smallest input
- A tie, tab or punctuation mark
- A result you can check by hand

Common mistakes

- Ignoring a method's return value
- Using split(' ') for all whitespace
- Replacing the first longest tie
- Dividing by zero words

Which test caught a mistake in your first attempt?

Questions?